

MALLA REDDY UNIVERSITY

SentinelAI: An AI-Powered Cyber Defense Platform for Honeypot-Driven Threat Detection, Correlation, Investigation and SOC Automation

Design & developed by

G VAISHNAV KUMAR	2311CS040053
G HARSHAL	2311CS040059
CH DEEPAK	2311CS040033

GUIDED BY

Dr. G. Latha

Associate Professor

Department of Computer Science & Engineering (Cyber Security)

IV YR - I SEM

Malla Reddy University, Hyderabad

2023-2027

MALLA REDDY UNIVERSITY
(Telangana State Private Universities Act No.13 of 2020 and G.O.Ms.No.14, Higher Education (UE) Department)

Department of Computer Science & Engineering (Cyber Security)

CERTIFICATE

This is to certify that the project report entitled "SentinelAI: An AI-Powered Cyber Defense Platform for Honeypot-Driven Threat Detection, Correlation, Investigation and SOC Automation", submitted by G Vaishnav Kumar ([Roll Number]) and team towards the partial fulfillment for the award of Bachelor's Degree in Cyber Security from the Department of Computer Science and Engineering, Malla Reddy University, Hyderabad, is a record of bonafide work done by the students. The results embodied in the work are not submitted to any other University or Institute for the award of any degree or diploma.

Internal Guide
Dr. G Latha
Associate Professor

Head of the department
Dr. G. Anand Kumar
CSE (Cybersecurity)

External Examiner

DECLARATION

We hereby declare that the project report "SentinelAI: An AI-Powered Cyber Defense Platform for Honeypot-Driven Threat Detection, Correlation, Investigation and SOC Automation" has been carried out by us and this work has been submitted to the Department of Computer Science Engineering (Cyber Security), Malla Reddy University, Hyderabad. We further declare that this project work has not been submitted in full or part for the award of any other degree in any other educational institutions.

Place:

Date:

G Vaishnav Kumar	2311cs040053
G Harshal	2311cs040059
Ch Deepak	2311cs040033

ACKNOWLEDGEMENT

We extend our sincere gratitude to all those who have contributed to the completion of this project report. Firstly, we would like to extend our gratitude to Dr. V. S. K. Reddy, Vice Chancellor, for his visionary leadership and unwavering commitment to academic excellence.

We would also like to express our deepest appreciation to our project guide, [Guide Name], [Designation], whose invaluable guidance, insightful feedback and unwavering support have been instrumental throughout the course of this project for successful outcomes.

We extend our gratitude to our PRC Convenor, [Name], for giving valuable inputs and timely guidelines to improve the quality of our project through a critical review process. We thank our project coordinator, [Name], for timely support.

We are also grateful to Dr. G. Anand Kumar, Head of the Department, Cyber Security, for providing us with the necessary resources and facilities to carry out this project.

We are deeply indebted to all of them for their support, encouragement and guidance, without which this project would not have been possible.

G Vaishnav Kumar 2311cs040053

G Harshal 2311cs040059

Ch Deepak 2311cs040033

ABSTRACT

Security teams are routinely overwhelmed by raw events: logs without context, alerts without relationships and tools that each solve only one part of the problem. SentinelAI addresses this gap with an AI-assisted cybersecurity monitoring, threat detection, investigation, incident correlation, active defense and reporting platform that simulates the workflow of a modern Security Operations Centre.

The platform collects events from a controlled honeypot environment (the Aetheris decoy services), simulated attacks, a web application firewall and authentication activity. Events are validated, classified by transparent rule-based detection for SQL injection, cross-site scripting, path traversal and brute-force activity, enriched with geolocation and reputation data, normalised into a common schema and correlated into incidents mapped to MITRE ATT&CK techniques. A real-time React dashboard with a 3D threat globe presents the results over WebSockets.

An AI Security Copilot built on Groq Cloud streams explanations through Server-Sent Events. A seven-action Investigator workspace provides incident analysis, severity explanation, IOC extraction, containment recommendations, MITRE mapping, attack timelines and executive summaries. Crucially, the language model never acts as the detector: deterministic rules identify the attack first and the model receives verified, structured context, which reduces the risk of unsupported conclusions. A deterministic fallback engine keeps the system functional when the API is unavailable.

The back end uses Python and FastAPI with SQLAlchemy persistence (SQLite for development and PostgreSQL for production), Argon2id password hashing, hashed session tokens, role-based access control, trusted-host and CORS protection, Docker packaging with Nginx, and a hybrid deployment model that keeps the honeypot lab separate from the public application. The system is verified by a 51-test automated Pytest suite and a successful production build. SentinelAI is intended for education, controlled laboratory experimentation, portfolio demonstration and SOC training, and is positioned as a foundation for future operational development rather than a replacement for enterprise platforms.

INDEX

CONTENTS **Page No.**

Chapter 1 Introduction....1
1.1 Problem Definition & Description....2
1.2 Objectives of the Project....2
1.3 Scope of the Project....3
Chapter 2 System Analysis....4
2.1 Existing System....4
2.1.1 Background & Literature Survey....4
2.1.2 Limitations of Existing System....5
2.2 Proposed System....6
2.2.1 Advantages of Proposed System....6
2.3 Software & Hardware Requirements....7
2.3.1 Software Requirements....7
2.3.2 Hardware Requirements....7
2.4 Feasibility Study....8
Chapter 3 Architectural Design....9
3.1 Modules Design....9
3.2 Method and Algorithm Design....10
3.2.1 Attack Detection Methods....10
3.2.2 Correlation Methods....11
3.2.3 Severity Classification....11
3.2.4 AI Investigation Method....12
3.2.4.1 The Seven Investigator Actions....12
3.3 Project Architecture....12
3.3.1 Architectural Diagram....12
3.3.2 Data Flow Diagram....13
3.3.3 Sequence Diagram....14
3.3.4 Security Lifecycle....15
3.3.5 Deployment Architecture....15
Chapter 4 Application Code....17
4.1 Coding Blocks (Frontend & Backend Code)....17
4.1.1 Repository Structure....17
4.1.2 Environment Configuration....17
4.1.3 Database Configuration Logic....17
4.1.4 Frontend Real-Time Client....18
4.1.5 Principal API Endpoints....18
4.1.6 Running the Application....18
4.1.7 Container Stack....19
4.2 Execution Flow....19
Chapter 5 Testing and Results....21
5.1 Resulting Screens....21
5.1.1 SOC Analyst Login Portal....21

5.1.2 Real-Time SOC Command Center....21
5.1.3 Live Threat Attack Feed & Triage....22
5.1.4 AI Security Copilot & Investigator....22
5.1.5 Threat Intelligence & Attacker Profiling....23
5.1.6 Active Defense WAF Manager....24
5.1.7 Aetheris Decoy Honeypot Lab....24
5.1.8 Executive Reports & Export....25
5.2 Test Results....25
5.2.1 Automated Backend Tests....25
5.2.2 Frontend Build Verification....26
5.2.3 Functional Test Cases....26
5.2.4 Evaluation Metrics....27
5.3 Result Analysis....27
5.3.1 Result Summary....27
5.4 Challenges Faced and Solutions....28
Chapter 6 Conclusion & Future Scope....29
6.1 Conclusion....29
6.2 Future Scope....29
Roadmap....30
References....31

LIST OF FIGURES

NAME OF THE FIGURE **Page No.**

Fig.1 Architectural Diagram of the SentinelAI Platform....12
Fig.2 Data Flow Diagram of the SentinelAI Platform....13
Fig.3 Sequence Diagram of the SentinelAI Platform....14
Fig.4 SentinelAI Security Lifecycle....15
Fig.5 Hybrid Deployment Architecture of SentinelAI....15
Fig.6 SOC Analyst Login Portal....21
Fig.7 Real-Time SOC Command Center (Dashboard)....22
Fig.8 Live Threat Attack Feed & Triage....22
Fig.9 AI Security Copilot & Incident Investigator....23
Fig.10 Threat Intelligence & Attacker Profiling....23
Fig.11 Active Defense Web Application Firewall (WAF) Manager....24
Fig.12 Aetheris Decoy Honeypot Lab....24
Fig.13 Executive Compliance & Incident Reports....25

CHAPTER - 1
INTRODUCTION

Modern information systems generate security-relevant data continuously. Every web request, login attempt, one-time-password submission, firewall decision and application log entry may carry evidence of an attack, yet raw logs on their own say very little. A single failed login is usually harmless; fifteen failed logins from one address suggest brute-force activity; fifteen failures followed by a success suggest that an account may have been compromised. A request containing an apostrophe is ordinary user input, but a request containing an apostrophe followed by an always-true condition and a SQL comment sequence is almost certainly an injection attempt. A security platform therefore has to do more than display logs: it must recognise patterns, add context, relate events to one another and help a human analyst decide what to do next.

Security Operations Centres (SOCs) exist to perform this work. SOC analysts rely on Security Information and Event Management (SIEM) platforms, intrusion detection systems, web application firewalls, threat-intelligence feeds and incident-response tooling. In mature organisations these systems cooperate, but enterprise products are costly, complex to deploy and difficult for students and small teams to understand. They also tend to separate detection, enrichment, investigation, response and reporting into different products, so the end-to-end workflow is rarely visible in one place.

SentinelAI is an autonomous, full-stack Security Operations Centre simulation, threat-intelligence correlation and incident-response platform designed to make that complete workflow visible and understandable. It combines a deception layer (the Aetheris decoy honeypot and an attack simulator), an active-defense web application firewall, rule-based attack detection, threat-intelligence enrichment, a correlation engine that groups micro-events into incidents mapped to MITRE ATT&CK, a streaming AI Security Copilot with a structured Investigator workspace, automated remediation playbooks, and PDF/CSV reporting, all presented through a real-time React dashboard.

The platform is built with a React 19 and Vite front end, a Python FastAPI back end, SQLAlchemy 2.0 persistence (SQLite for development and PostgreSQL for production), WebSockets for live event delivery, Server-Sent Events for streaming AI responses and Groq Cloud for large-language-model inference. Its guiding principle is that the language model never acts as the detector: deterministic rules detect and classify an attack first, and the AI is then supplied with verified, structured context so that it can explain, map and recommend. This keeps the system explainable and reduces the risk of hallucinated conclusions.

By carrying an event through the lifecycle Collect → Detect → Enrich → Normalize → Correlate → Investigate → Respond → Report inside a single application, SentinelAI demonstrates how a security team can turn low-level telemetry into understandable incidents, make defensive decisions and document the outcome. It is intended for cybersecurity

education, controlled laboratory experimentation, academic demonstration and SOC training, and as a foundation for future small-scale operational monitoring.

1.1 Problem Definition & Description

Security teams and learners face several connected problems that existing tools address only in part:

- **Alert volume and fatigue.** Monitoring tools can raise hundreds or thousands of alerts a day, many of them low risk, duplicated or unrelated. Analysts spend time on noise instead of real incidents.
- **Lack of context.** A raw log line contains a timestamp, an address, an endpoint, a status code and a payload, but does not explain the attack technique, the likely objective, the severity or the recommended response.
- **Delayed correlation.** Attackers rarely use a single vector. Brute-force attempts on an administrative portal often precede SQL injection or lateral movement, and correlating such events manually can take hours.
- **Fragmented tooling.** Detection, threat intelligence, firewall management, incident handling, reporting and AI assistance usually live in different products, forcing analysts to move between interfaces and combine results by hand.
- **Passive defense.** Traditional firewalls drop traffic without learning anything about the attacker's technique, payloads or persistence, so valuable intelligence is lost.
- **Limited training environments.** Students may learn individual tools such as Nmap, Wireshark or Burp Suite but rarely see how security events flow through an end-to-end SOC pipeline.

There is therefore a clear need for an integrated platform that captures events from controlled sources, detects and classifies attacks, enriches and correlates them into incidents, explains them with AI using verified context, supports defensive actions and produces audit-ready reports, while remaining small enough to be understood, demonstrated and extended.

1.2 Objectives of the Project

The primary objective of SentinelAI is to build an integrated cybersecurity operations platform that collects, detects, enriches, correlates, investigates, responds to and reports on security events. The specific objectives are:

- **Real-time security visibility:** provide a live SOC command centre with threat-level indexing, host vitals, a 3D global threat view and a WebSocket-driven attack feed.
- **Deception-based telemetry:** deploy the Aetheris decoy honeypot (HTTP, SSH, FTP and Telnet decoys) and an attack simulator so that realistic attack activity can be generated and captured safely in a controlled laboratory.

- **Accurate, explainable detection:** implement transparent, rule-based detection for SQL injection, cross-site scripting, path traversal, OTP brute force and authentication brute force.
- **Threat-intelligence enrichment:** attach geolocation, ISP, ASN and abuse-reputation context to source addresses, with a deterministic fallback so the pipeline never fails when external services are unavailable.
- **Incident correlation:** normalise events from different sources into a common schema and correlate them into incidents (brute force followed by success, lateral movement, escalation) mapped to MITRE ATT&CK techniques.
- **AI-assisted investigation:** integrate a streaming AI Security Copilot and an Investigator workspace with seven structured actions that operate on verified attack and incident context.
- **Active defense and response:** provide a WAF manager with rules, IP quarantine and 403 blocking, plus automated remediation playbooks and incident response actions.
- **Reporting and auditability:** generate executive PDF reports and CSV telemetry exports, and persist every record in a relational database.
- **Production-oriented foundation:** support role-based access control, secure session handling, PostgreSQL, Docker packaging and a hybrid public/private deployment model.

1.3 Scope of the Project

The scope of SentinelAI covers web-application security monitoring, honeypot and simulated attack activity, IP-based threat intelligence, SQL injection, XSS and path traversal detection, OTP and authentication brute-force detection, WAF event management, incident correlation, AI-assisted investigation, remediation playbooks, report generation and deployment preparation. It is aimed at cybersecurity students, SOC trainees, project evaluators, security researchers working in controlled environments and small technical teams who need an understandable end-to-end SOC workflow.

SentinelAI does not attempt to replace mature enterprise SIEM or XDR platforms. It is not designed for full packet inspection, endpoint telemetry from thousands of devices, large-scale log ingestion, multi-region high availability or regulated production environments. These are treated as future expansion areas. The table below summarises what is currently inside and outside the scope.

In scope (current)	Out of scope / future
Honeypot and simulator event capture	Enterprise-scale log ingestion
Rule-based web attack detection (SQLi, XSS, traversal, brute force)	Machine-learning anomaly detection
IP enrichment with deterministic fallback	Full packet inspection (Zeek / Suricata integration)

In scope (current)	Out of scope / future
Correlation into incidents with MITRE mapping	Endpoint agents and eBPF sensors
Streaming AI Copilot and Investigator workspace	Multiple LLM providers and local model fallback beyond rule engine
WAF manager, IP quarantine, playbooks	Cloud-firewall webhooks, ticketing and chat integrations
PDF / CSV reporting, RBAC, session security	Multi-tenant workspaces, compliance templates
Docker / PostgreSQL / hybrid deployment foundation	High-availability, public production hardening

CHAPTER - 2
SYSTEM ANALYSIS

2.1 Existing System

Existing security monitoring is built from several mature but separate technologies. SIEM platforms centralise logs for search and alerting; intrusion detection and prevention systems match traffic against signatures; web application firewalls filter HTTP requests; honeypots capture attacker behaviour; threat-intelligence services score indicators; and SOAR products automate response playbooks. Each is effective within its own domain, but the analyst is left to connect them manually.

In practice an analyst reads an alert, copies an address into an intelligence website, searches logs for related activity, decides whether the events belong to one campaign, writes a firewall rule, records the case in a ticket and finally drafts a report. This workflow is slow, repetitive and heavily dependent on individual expertise, and it is difficult to reproduce in a classroom or laboratory.

2.1.1 Background & Literature Survey

Security Information and Event Management (SIEM)

SIEM systems such as Splunk Enterprise Security, IBM QRadar, Microsoft Sentinel, Elastic Security and LogRhythm collect logs from servers, firewalls, endpoints, cloud services and authentication systems, then parse, normalise, index and correlate them. Their main advantage is centralised visibility across data sources. Their limitations are deployment effort, specialised query languages, licensing cost, continuous rule tuning and a tendency to generate many false positives when detection logic is not carefully configured. SentinelAI adopts the SIEM concepts of normalised logs, detection rules, correlation, incidents, dashboards and reporting in a much smaller and more educational architecture.

Intrusion Detection and Prevention Systems (IDS/IPS)

Snort, Suricata and Zeek inspect network traffic using signatures or anomaly models. Signature-based detection is explainable and effective against known attacks but may miss new or modified ones; anomaly-based detection can find unknown behaviour but produces false positives. SentinelAI currently relies on rule-based and pattern-based logic for explainability and controlled testing, while its architecture leaves room for anomaly detection later.

Honeypots

A honeypot is a deliberately exposed decoy system used to observe attacker behaviour. Low-interaction honeypots emulate selected services safely but yield limited information; high-interaction honeypots provide richer data but create the risk that a compromised decoy is used to attack other systems. Cowrie, Dionaea, Honeyd and OpenCanary are well-known examples. The SentinelAI Aetheris lab uses passive, protocol-level decoys (HTTP, SSH, FTP and Telnet) designed for payload inspection, and keeps the lab separable from the public application.

Web Application Firewalls (WAF)

ModSecurity, Cloudflare WAF, AWS WAF and Azure WAF inspect paths, headers, query strings and bodies against rules. They block known malicious patterns but badly designed rules produce false positives. SentinelAI includes a WAF manager with rules, hit counters, IP quarantine and a controlled 403 response, and mirrors every WAF decision into the attack feed so the analyst keeps full visibility.

Threat Intelligence Platforms

AbuseIPDB, VirusTotal, AlienVault OTX and MISP provide reputation data for indicators. Such data improves context but is not proof: an address may be shared, reassigned or used by a VPN. SentinelAI treats enrichment as supporting evidence and keeps the decision with the analyst.

Security Orchestration, Automation and Response (SOAR)

Cortex XSOAR, Splunk SOAR and IBM Security SOAR execute playbooks that gather intelligence, block addresses and open tickets. SentinelAI provides a lightweight equivalent through incident actions, remediation playbooks (such as Brute Force IP Containment and SQL Injection Quarantine), notification hooks and AI-generated recommendations.

AI-Assisted SOC Systems

Generative AI is increasingly used to explain alerts, summarise incidents and draft queries. The benefit is reduced analyst workload; the risk is that language models can state unsupported conclusions. SentinelAI reduces this risk by running deterministic detection first and then injecting structured attack and incident context (severity, payload, protocol, source, timeline) into the AI request, and by presenting the output as a recommendation that requires analyst review.

Comparison with existing system types

System type	Main strength	Limitation	SentinelAI approach
SIEM	Centralised log search and correlation	Expensive and complex	Smaller integrated SOC workflow
IDS / IPS	Real-time signature detection	Limited investigation context	Detection linked to AI and incident views
Honeypot	Captures attacker behaviour	Requires isolation	Integrated, controlled Aetheris Honeypot Lab
WAF	Blocks malicious HTTP traffic	False positives possible	Rule management with event visibility
Threat intelligence	Adds external reputation data	Data may be incomplete	External enrichment with deterministic fallback
SOAR	Automates response workflows	Complex integration	Lightweight playbooks and incident actions
AI copilot	Fast explanation and summaries	Hallucination risk	Context-grounded prompts with analyst review

2.1.2 Limitations of Existing System

- **High cost and complexity:** enterprise SIEM/SOAR products require licensing, infrastructure and specialist skills, which puts them out of reach for students and small teams.
- **Alert overload with little context:** tools raise many isolated alerts without explaining technique, objective or next step.
- **Fragmented workflow:** detection, intelligence, firewalling, incident tracking, AI assistance and reporting are spread across separate interfaces.
- **Passive defense:** conventional firewalls block traffic but do not capture the attacker's payloads and behaviour for analysis.
- **Limited AI grounding:** general-purpose chatbots answer security questions without access to the actual attack records, so answers may be generic or wrong.
- **Poor teaching value:** commercial platforms hide their internals, so learners cannot inspect how events become incidents.

Overall, existing approaches are less effective at giving small teams and learners a complete, understandable and affordable view of the security lifecycle.

2.2 Proposed System

The proposed system, SentinelAI, is a unified AI-assisted SOC platform. Events from the Aetheris honeypot, the attack simulator, the WAF and authentication activity enter a single ingestion pipeline. Each event is validated, passed to the detection engine, stored as a structured attack record, enriched with threat-intelligence data, normalised into a common log schema and evaluated by the correlation engine. When a rule matches, an incident is created or updated and the event is broadcast over WebSockets so that the dashboard updates without a page refresh.

The analyst can then open any attack or incident in the AI Investigator. The back end injects the selected record into the AI prompt and streams the response using Server-Sent Events. Seven structured actions (Analyze Incident, Explain Severity, Extract IOCs, Recommend Containment, Map to MITRE ATT&CK, Generate Attack Timeline and Draft Executive Summary) turn technical evidence into understandable guidance. The analyst can respond by blocking an address, adding a WAF rule, running a playbook, escalating or resolving the incident, and finally export a PDF or CSV report.

A secure application foundation supports this workflow: Argon2id password hashing, hashed server-side session tokens delivered in HttpOnly cookies, role-based access control for administrators and analysts, trusted-host and CORS protection, environment-based secrets, dual-dialect database support and a decoupled deployment model (Vercel front end, Railway back end and PostgreSQL, with the honeypot kept in a private lab).

2.2.1 Advantages of Proposed System

- **End-to-end lifecycle in one application:** collection, detection, enrichment, normalisation, correlation, investigation, response and reporting are connected rather than separate.
- **Explainable detection:** transparent rules produce an attack type, severity, rule identifier and evidence for every alert.
- **Context-grounded AI:** the model only explains what the deterministic engine has already verified, which lowers hallucination risk.
- **Real-time visibility:** WebSocket streaming updates counters, charts and feeds as events arrive.
- **Resilience:** the AI Copilot falls back to a deterministic local engine when the API key is missing or rate-limited, and threat-intelligence lookups fall back to deterministic enrichment.
- **Active defense with intelligence gathering:** honeypots and WAF quarantine both block and observe attackers.
- **Educational value:** the architecture is small enough to inspect, demonstrate and extend, and is safe to run in a local laboratory.
- **Auditable records:** attacks, incidents, WAF hits, playbook runs and reports are persisted with timestamps for later review.

2.3 Software & Hardware Requirements

2.3.1 Software Requirements

Category	Technology	Purpose
Operating system	Windows, Linux, macOS	Development and deployment host
Back-end language / framework	Python 3.11+, FastAPI, Uvicorn	REST API, WebSockets, SSE, middleware
Data layer	SQLAlchemy 2.0, Pydantic v2	ORM models and request validation
Database	SQLite (dev), PostgreSQL (prod)	Persistent storage
Front-end	React 19, Vite 8, Lucide React	Single-page application
Visualisation	Recharts, Three.js	Charts and 3D threat globe
Real-time transport	WebSockets, Server-Sent Events	Live feeds and streamed AI output
AI provider	Groq Cloud (openai/gpt-oss-120b default; Llama 3.3 70B Versatile in earlier builds)	Security reasoning and reporting
Threat intelligence	ip-api.com, AbuseIPDB (optional key)	Geolocation and reputation

Category	Technology	Purpose
Security	Argon2id, SHA-256 token hashing, TrustedHost, CORS	Authentication and request protection
Containers / proxy	Docker, Docker Compose, Nginx	Packaging and reverse proxying
Hosting targets	Vercel (front end), Railway (back end + PostgreSQL)	Decoupled deployment
Testing / tooling	Pytest, ESLint, Git, GitHub	Verification and version control
Browser	Google Chrome, Mozilla Firefox, Microsoft Edge	Client access

2.3.2 Hardware Requirements

Component	Recommendation
Processor	Intel Core i5 or higher multi-core processor
RAM	Minimum 8 GB (Docker stack, database and development servers)
Storage	Minimum 20 GB free space (images, database, reports, backups)
Network	Internet access for Groq API and threat-intelligence lookups; local network for LAN honeypot mode
Display	Standard monitor with good resolution (dashboard tested at 100 percent browser zoom)
Input devices	Keyboard and mouse

2.4 Feasibility Study

The feasibility study concludes that SentinelAI is technically, economically, operationally, legally and temporally feasible. It is built entirely on open-source and widely supported technologies and can be developed, deployed and maintained by a small team.

1. Technical Feasibility

Python, FastAPI, React, Vite, SQLAlchemy, PostgreSQL, Docker and Groq Cloud are mature, well documented and widely available. The working prototype demonstrates the feasibility of real-time WebSocket streaming, SSE-based AI responses, rule-based detection and correlation, and the 51-test automated suite together with a successful production build show that the implementation is stable.

2. Economic Feasibility

Development relies on open-source frameworks and free or pay-as-you-go services (SQLite locally, Groq API, free-tier hosting for the front end and back end). No specialised security appliances are required, so development and operating costs are low compared with commercial SIEM licensing.

3. Operational Feasibility

The dashboard presents alerts, incidents, AI analysis and reports in one interface with quick-action buttons, so a trainee analyst can use the system without deep AI expertise. Demonstration scripts and a guided workflow further reduce the learning curve.

4. Legal and Ethical Feasibility

SentinelAI is designed for controlled laboratory use. The honeypot and simulator must be operated only on systems and networks the user owns or is authorised to use, honeypots must contain no real credentials or sensitive data, and any public exposure must follow hardened deployment practice. Third-party data (for example IP intelligence) is used according to the providers' terms, and secrets are kept in environment variables rather than source control.

5. Time Feasibility

The project was delivered incrementally through well-defined phases: backend and dashboard, honeypot and detection, WAF and active defense, threat intelligence, correlation, incident response, AI integration and Investigator workspace, and finally the deployment foundation. Remaining work such as local Docker validation, migrations and security hardening can be scheduled as further phases without redesigning the core.

CHAPTER - 3
ARCHITECTURAL DESIGN

3.1 Modules Design

SentinelAI is organised into loosely coupled modules, each with defined inputs and outputs, mirroring how a real SOC divides responsibilities between collection, detection, intelligence, investigation and response.

1. Security Event Ingestion Module

Receives honeypot requests, simulator events, authentication records, WAF matches and administrative activity. It extracts timestamp, source IP, endpoint, HTTP method, payload, headers, protocol, user agent, response status and source module, validates the fields and produces a structured raw event.

2. Detection Engine

Applies transparent pattern and threshold rules to the raw event. Output: attack classification, severity, detection reason, rule identifier and evidence. Covered categories include SQL injection, cross-site scripting, path traversal, OTP brute force and authentication brute force.

3. Threat Intelligence Engine

Enriches the source IP with country, region, ISP/ASN, organisation, abuse score and classification using ip-api.com and, when a key is configured, AbuseIPDB. A deterministic fallback guarantees that enrichment never breaks the pipeline.

4. Normalisation Module

Maps events from different sources into a common NormalizedLog schema (source, event type, source IP, destination, timestamp, severity, action, user, protocol, payload) so that heterogeneous events can be compared and correlated.

5. Correlation Engine

Searches normalised logs and historical attacks for relationships (same source, target, account, attack type, close timestamps, failure-then-success, multiple destinations) and creates or updates correlated incidents mapped to MITRE ATT&CK techniques.

6. Incident Management & Response Module

Maintains the incident lifecycle (open, investigating, contained, resolved, escalated), supports attack-level actions (acknowledge, investigate, escalate, contain, block, resolve, notify, add note) and exposes webhook-style notifications.

7. AI Security Copilot & Investigator

Sends structured context (selected attack or incident, payload, severity, protocol, threat intelligence, timeline) to Groq Cloud through the back end and streams the answer via SSE. Provides seven investigation actions and a deterministic fallback engine. The API key never leaves the back end.

8. Active Defense WAF Manager

Stores WAF rules and hits, evaluates requests against active rules, enforces temporary or permanent IP quarantine, returns a controlled 403 response and records every decision for dashboard visibility.

9. Aetheris Honeypot Lab

Runs decoy services (HTTP portal on 8088, SSH on 2222, FTP on 2121, Telnet on 2323) with a toggle between loopback (127.0.0.1) and LAN (0.0.0.0) binding, a live activity log and a PowerShell firewall-rule generator.

10. Decoy Malware Sandbox

Accepts suspicious artifacts into an isolated, non-executable staging directory, computes MD5, SHA-1 and SHA-256 hashes and derives a static heuristic score from headers, strings and entropy.

11. Playbook Engine

Executes pre-configured remediation workflows such as Brute Force IP Containment and SQL Injection Quarantine and logs each step, impacted asset and mitigation outcome.

12. Reporting Module

Transforms attack, incident, intelligence and action data into executive PDF reports and CSV telemetry exports, stored on a persistent path.

13. Authentication, RBAC & Admin Module

Handles registration and login with Argon2id, issues session cookies, enforces admin and analyst roles through FastAPI dependencies and records administrative and suspicious activity.

14. WebSocket Manager

Broadcasts newly created attacks and incidents to connected dashboards on /api/attacks/ws with automatic client reconnection.

15. Database Module

SQLAlchemy 2.0 models (22 entities) for attacks, incidents, normalised logs, WAF rules and hits, users and sessions, reports, settings and playbooks, with dual SQLite/PostgreSQL support and idempotent column creation at start-up.

3.2 Method and Algorithm Design

This section describes the method and step-by-step algorithm used by each processing stage. The guiding design rule is: **Rule Engine → Verified Security Context → AI Explanation.** The AI is never the sole detector.

3.2.1 Attack Detection Methods

SQL injection. The detector examines the decoded payload for combinations of indicators: quote characters used to terminate input, always-true conditions, SQL comment sequences,

UNION SELECT patterns, database metadata references and time-delay functions. A single apostrophe is deliberately not sufficient, because legitimate names may contain one.

Cross-site scripting. Raw and URL/HTML-decoded content is searched for script tags, javascript: URLs, event handlers such as onerror, SVG-based execution, iframe injection and encoded browser-executable content.

Path traversal. Request paths and parameters are checked for directory-traversal sequences (including encoded variants) and attempts to reach sensitive files.

OTP and authentication brute force. Failures are counted per source IP and/or account inside a sliding time window. When the configurable threshold is exceeded an attack record is created; if a success follows the failures, severity is escalated and a possible account-compromise incident is raised.

Algorithm 1 — Detection pipeline (pseudocode)

```
function process_event(event):
    if not validate(event): reject_or_normalise(event)
    if waf.is_quarantined(event.source_ip):
        return http_403(event)                # active defense
    result = detect(event)                    # rules + thresholds
    if result.is_attack:
        attack = store(event, result)         # attack record + id
        attack.intel = enrich(event.source_ip)   # geo, ASN, abuse score
        log = normalise(attack)               # common schema
        incident = correlate(log)             # may create/update incident
        broadcast_websocket(attack, incident)
    return result
```

3.2.2 Correlation Methods

Rule	Input pattern	Output
Brute force → success	Many failures from one source/account followed by a success within the window	Possible account compromise incident (severity escalated)
Lateral movement	One source touching multiple internal targets with repeated service or authentication activity	Potential lateral movement incident
Escalation chain	Low-severity reconnaissance → exploitation attempt → privileged or sensitive action	High-severity escalation incident
Multi-vector campaign	Same source performing XSS probing, OTP brute force and admin-path scanning	Single correlated high-severity incident

3.2.3 Severity Classification

Level	Typical criteria
Low	Reconnaissance or isolated suspicious activity
Medium	Repeated or credible attack attempt
High	Strong exploitation evidence or repeated malicious activity

Level	Typical criteria
Critical	Confirmed compromise, successful escalation or coordinated attack sequence

Severity is derived from attack type, number of attempts, target sensitivity, threat reputation, compromise indicators, correlation strength and repetition. Attacker dossiers additionally receive a 0–100 risk score based on attack frequency, payload severity and multi-vector targeting.

3.2.4 AI Investigation Method

1. The analyst selects an attack or incident (for example via /agent?analyze_attack=<id>).
2. The back end loads the full record: attack type, payload, severity, protocol, source information, threat intelligence and incident timeline.
3. A system prompt containing security instructions, the structured context and output-format requirements is built on the server.
4. The request is sent to Groq Cloud; the response is streamed back through SSE and rendered progressively. Default settings favour focused answers (temperature 0.2 with a modest token limit).
5. If the API key is missing or the provider is rate-limited, the deterministic local fallback engine produces the investigation output.
6. The analyst reviews the explanation, MITRE mapping, IOCs and recommended containment, then chooses a response action.

3.2.4.1 The Seven Investigator Actions

#	Action	Result
1	Analyze Incident	What happened, how it works and why it is dangerous
2	Explain Severity	Justification of the assigned severity level
3	Extract IOCs	IP addresses, payload fragments, user agents, endpoints, time range
4	Recommend Containment	Immediate blocking and WAF-rule suggestions to be reviewed before use
5	Map to MITRE ATT&CK	Techniques such as T1110 Brute Force, T1190 Exploit Public-Facing Application, T1059 Command and Scripting Interpreter
6	Generate Attack Timeline	Ordered sequence of related events
7	Draft Executive Summary	Non-technical summary of impact, actions and recommendations

3.3 Project Architecture

3.3.1 Architectural Diagram

SentinelAI follows a layered client–server architecture with event-driven real-time communication. Fig. 1 shows the presentation tier, ingress layer, application tier, data tier, external services and the private-lab event sources.

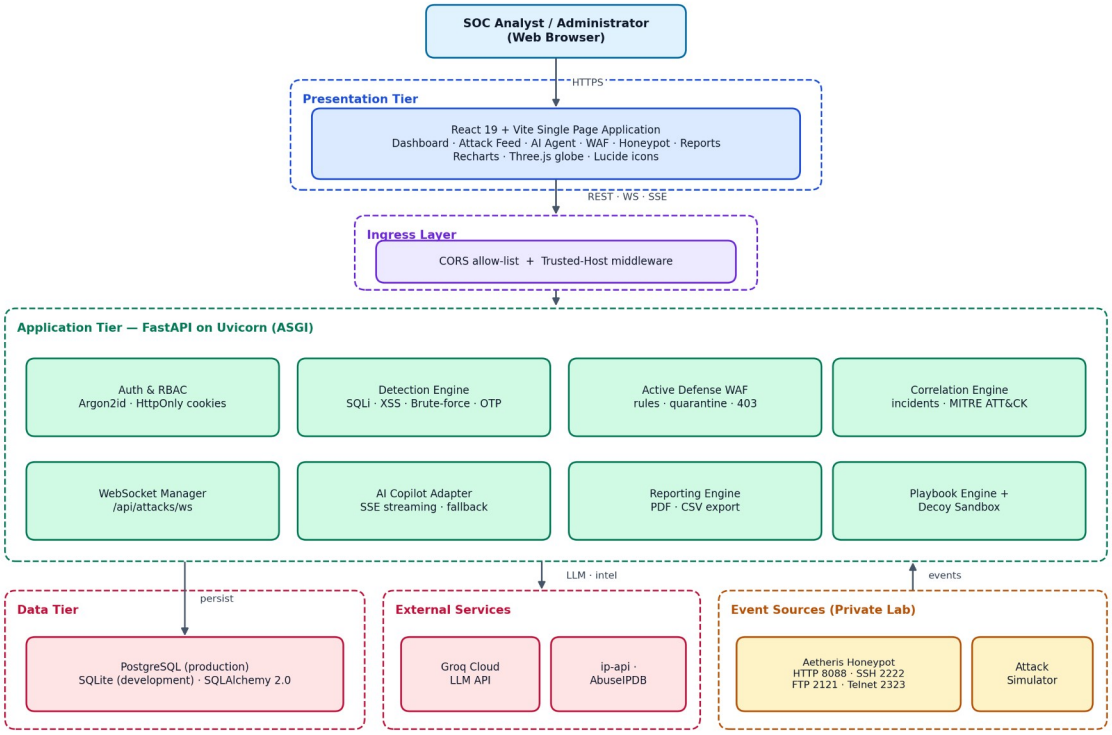


Fig. 1 Architectural Diagram of the SentinelAI Platform

Presentation layer. A React 19 and Vite single-page application renders dashboards, tables, charts, the 3D globe, investigation views and administrative pages. It talks to the back end only through REST, WebSocket and SSE; it never connects directly to the database, Groq or the intelligence providers.

Ingress layer. Starlette TrustedHostMiddleware validates the Host header and a strict, credentialed CORS allow-list prevents arbitrary web origins from issuing authenticated requests.

Application layer. FastAPI on Uvicorn hosts authentication and RBAC, the detection engine, WAF, correlation engine, WebSocket manager, AI adapter, reporting engine, playbook engine and decoy sandbox, acting as the central controller between the front end and internal modules.

Data and external layer. SQLAlchemy persists records in PostgreSQL (production) or SQLite (development). Groq Cloud supplies inference and ip-api/AbuseIPDB supply IP reputation.

Event sources. The Aetheris honeypot and the attack simulator submit events to the ingestion API. In the hybrid model these run in a private lab and connect to the public back end through an authenticated channel.

3.3.2 Data Flow Diagram

Fig. 2 shows how data moves between the analyst, the attacker or simulator, the security administrator, the platform processes, the external services and the data stores.

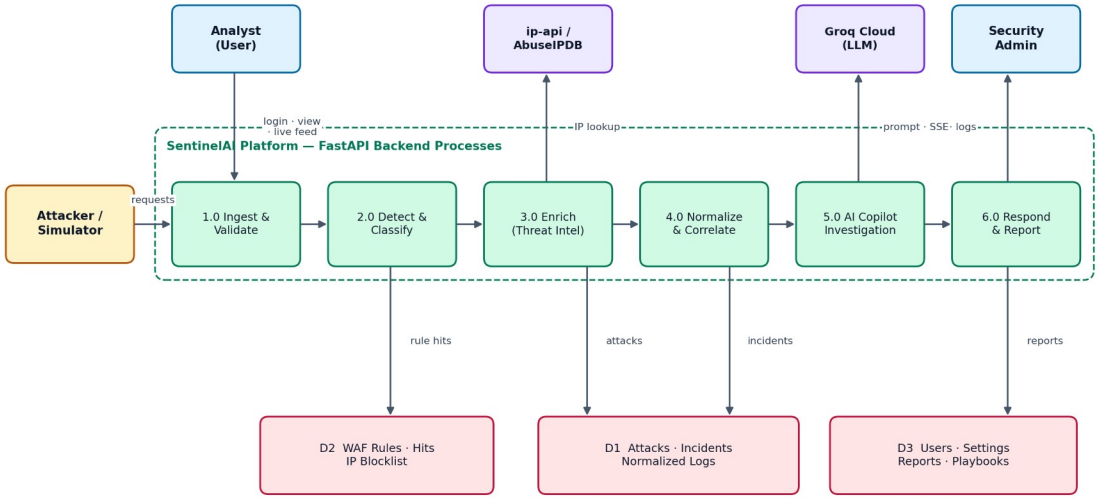


Fig. 2 Data Flow Diagram of the SentinelAI Platform

Malicious requests enter process 1.0 where they are validated; process 2.0 detects and classifies them (consulting WAF rules and the quarantine list); process 3.0 enriches the source IP using external intelligence; process 4.0 normalises the event and correlates it into incidents; process 5.0 serves AI-assisted investigation by sending verified context to Groq Cloud; and process 6.0 handles response actions and report generation. Attacks, incidents and normalised logs are held in D1, WAF rules, hits and the blocklist in D2, and users, settings, reports and playbooks in D3. The analyst receives live feeds and reports; the administrator receives logs and status.

3.3.3 Sequence Diagram

The sequence diagram in Fig. 3 shows one complete cycle from a malicious request to an AI-assisted response.

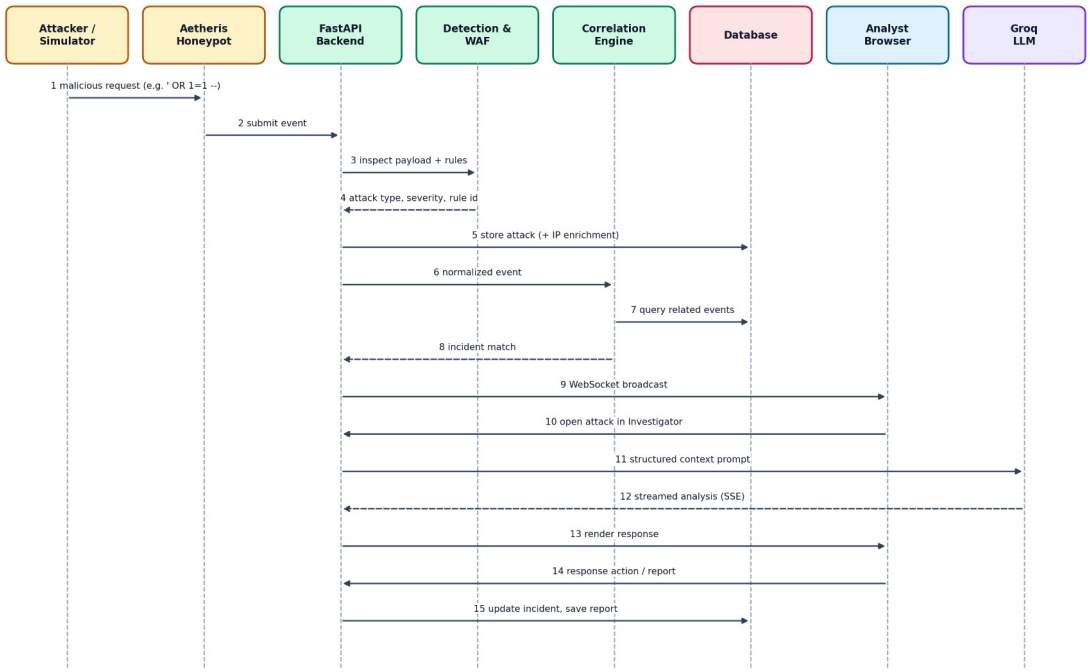


Fig. 3 Sequence Diagram of the SentinelAI Platform

The request reaches the honeypot, which forwards it to the back end. The detection engine and WAF return an attack type, severity and rule identifier; the attack is stored with IP enrichment; the normalised event is passed to the correlation engine, which queries related events and returns an incident match; the result is broadcast to the dashboard over WebSocket. The analyst opens the attack in the Investigator, the back end sends structured context to the LLM, the response is streamed back, and the analyst's response action updates the incident and report data in the database.

3.3.4 Security Lifecycle

Fig. 4 summarises the lifecycle implemented by the platform, from collection through reporting.

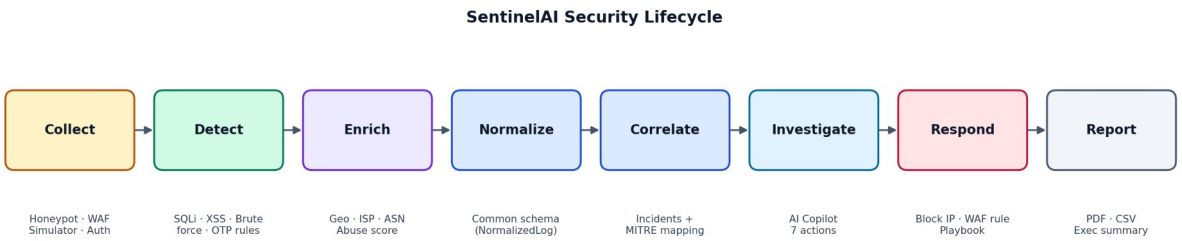


Fig. 4 SentinelAI Security Lifecycle

3.3.5 Deployment Architecture

SentinelAI is architected for a hybrid, decoupled deployment (Fig. 5). The React front end is served as a static build from the Vercel edge CDN with SPA rewrites. The FastAPI back end

runs as a persistent container built from Dockerfile.backend (python:3.11-slim) on Railway, exposing a /ready probe that includes a database connectivity check and hosting the in-memory WebSocket hub. PostgreSQL runs as a managed service reachable only over Railway's private network. Honeypots, simulators and experimental listeners are intentionally kept in a private laboratory and are meant to connect to the public back end through an authenticated private tunnel (Tailscale or WireGuard), creating a trust boundary between public and high-risk components. A Docker Compose stack (PostgreSQL 15, back end, Nginx-served front end), backup and restore scripts are also provided for local container validation.

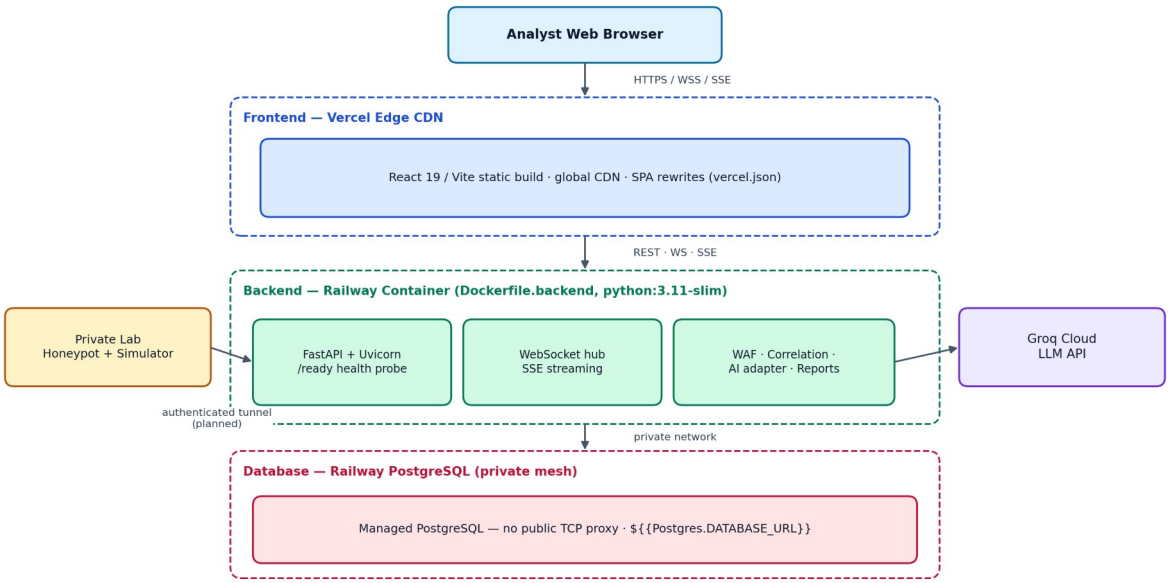


Fig. 5 Hybrid Deployment Architecture of SentinelAI

> *Deployment status: the production deployment configuration is in active preparation. Local Docker validation and the private-lab tunnel are planned next steps and are described honestly in Chapter 6.*

CHAPTER - 4
APPLICATION CODE

4.1 Coding Blocks (Frontend & Backend Code)

This section presents representative excerpts that describe how the SentinelAI implementation is organised and configured. The full source is available in the project repository at https://github.com/Vaishnav53/SentinelAI.

4.1.1 Repository Structure

Project layout

```
SentinelAI/
├── backend/                    # FastAPI application
│   ├── api/        agent.py · attacks.py · auth.py · health.py
│   │               reports.py · sandbox.py · waf.py
│   ├── core/       config.py · errors.py · logging_config.py · security.py
│   ├── database/   session.py            # engine, pooling, seed data
│   ├── models/                           # 22 SQLAlchemy models
│   ├── schemas/                          # Pydantic v2 schemas
│   ├── services/   active_defense.py · auth.py · correlation_engine.py
│   │               decoy_sandbox.py · honeypot.py · playbook_engine.py
│   └── tests/                            # Pytest suite (51 tests)
├── frontend/                   # React 19 / Vite SPA
│   └── src/  api/ · components/ · context/ · layouts/ · pages/ · routes/
├── docs/screenshots/           # application screenshots
├── Dockerfile.backend · Dockerfile.frontend · docker-compose.yml · nginx.conf
└── LICENSE · README.md · SECURITY.md
```

4.1.2 Environment Configuration

Sensitive values are read from environment files and are never committed. The back end validates them through Pydantic settings; production guards are activated when APP_ENV is set to production.

backend/.env (excerpt)

```
APP_ENV=development
APP_HOST=127.0.0.1
APP_PORT=8000
DATABASE_URL=sqlite:///./storage/sentinelai.db  # PostgreSQL in production
FRONTEND_ORIGIN=http://localhost:5173
TRUSTED_HOSTS=127.0.0.1,localhost
AUTH_COOKIE_SECURE=false                # true in production (HTTPS only)
AUTH_COOKIE_SAMESITE=lax                # none for cross-origin production
GROQ_API_KEY=<kept secret>              # optional: fallback engine if absent
DEFAULT_GROQ_MODEL=openai/gpt-oss-120b
LOG_LEVEL=INFO
```

4.1.3 Database Configuration Logic

The database layer reads DATABASE_URL. SQLite URLs receive SQLite-compatible connection options; PostgreSQL URLs receive pooling and production-compatible behaviour, and legacy postgres:// URLs are normalised to postgresql://. On start-up the application introspects the schema and idempotently creates missing columns, so updates apply smoothly across both dialects.

4.1.4 Frontend Real-Time Client

The front end derives the WebSocket address from an environment variable, or from the current page location with protocol-aware ws/wss selection, avoiding hard-coded localhost addresses that fail in production.

WebSocket connection logic (conceptual)

```
const wsBaseUrl =
  import.meta.env.VITE_WS_BASE_URL ||
  `${window.location.protocol === "https:" ? "wss" : "ws"}://${window.location.host}`;

const socket = new WebSocket(`${wsBaseUrl}/api/attacks/ws`);
socket.onmessage = (msg) => {
  const event = JSON.parse(msg.data);   // new attack or incident
  updateDashboardAndFeed(event);        // counters, charts, feed
};
```

4.1.5 Principal API Endpoints

Endpoint	Type	Purpose
/api/attacks/ws	WebSocket	Live attack and incident broadcast to dashboards
/api/agent/chat/stream	SSE	Streamed AI Copilot / Investigator response
/api/agent/chat	REST	Non-streaming AI chat
/api/attacks/{id}/incident-action	REST	Acknowledge, investigate, escalate, contain, block, resolve, notify
/agent?analyze_attack=<id>	UI route	Open an attack directly in the AI Investigator
/api/reports	REST	Generate and download PDF / CSV reports
/ready	REST	Readiness probe with database connectivity check

4.1.6 Running the Application

Backend (Terminal 1)

```
cd backend
python -m venv .venv && source .venv/bin/activate
# Windows PowerShell: .\.venv\Scripts\Activate.ps1
pip install -r requirements.txt && cp .env.example .env
python -m uvicorn backend.main:app --host 127.0.0.1 --port 8000 --reload
```

Frontend (Terminal 2)

```
cd frontend
npm install && cp .env.example .env
npm run dev          # open http://localhost:5173
```

4.1.7 Container Stack

The Docker Compose stack defines three services: db (postgres:15-alpine with a persistent volume, health check and environment-supplied password), backend (python:3.11-slim running Uvicorn on port 8000, with a persistent reports volume) and frontend (a multi-stage Node build served by Nginx). Nginx serves the React build, proxies /api/ to FastAPI, supports

WebSocket upgrades on /api/attacks/ws, enforces a maximum request-body size and falls back to index.html for client-side routing. Helper scripts db_backup.sh and db_restore.sh run pg_dump and restore a chosen SQL file respectively.

Backend database URL inside Compose

```
DATABASE_URL=postgresql://postgres:${POSTGRES_PASSWORD}@db:5432/sentinelai
```

4.2 Execution Flow

1. Start-up and Initialisation

The back end loads configuration, initialises the engine and creates or migrates tables, seeds initial data, bootstraps the administrator account, registers routers and middleware (CORS and trusted hosts) and starts the WebSocket manager and the background simulator loop.

2. Authentication

The analyst signs in. The server verifies the Argon2id hash, creates a 32-byte random session token, stores only its SHA-256 hash and returns the token in an HttpOnly, Secure, SameSite cookie. FastAPI dependencies enforce admin and analyst roles.

3. Dashboard Initialisation

The front end fetches current metrics, host vitals and threat level, and opens the WebSocket connection with automatic reconnection.

4. Event Arrival

A honeypot interaction, simulator event or WAF match reaches the ingestion API.

5. Detection and Enrichment

The detection engine classifies the event; the source IP is enriched with geolocation, ASN and abuse reputation, using the deterministic fallback if providers fail.

6. Storage, Normalisation and Correlation

The attack record is stored, normalised and evaluated by the correlation engine, which creates or updates an incident and maps it to MITRE ATT&CK.

7. Real-Time Broadcast

The WebSocket manager pushes the new attack or incident to all connected clients; counters, charts, the feed and the globe update without refresh.

8. Triage and AI Investigation

The analyst opens the attack or incident in the Investigator; the server injects the structured context and streams the AI answer; the analyst runs the seven actions as needed.

9. Response

The analyst blocks the IP, adds a WAF rule, runs a playbook, escalates, contains or resolves the incident; every action is recorded.

10. Reporting and Retention

An executive PDF or CSV report is generated and stored; all records remain in the database for audit, and backups protect production data.

CHAPTER - 5
TESTING AND RESULTS

5.1 Resulting Screens

> *Screenshot slots below are placeholders. Replace each with the matching image from the repository folder docs/screenshots/ (right-click the image in Word, choose Change Picture), keeping the caption.*

5.1.1 SOC Analyst Login Portal

The entry point to the protected environment. Credentials are sent to the back end, where Argon2id password verification is performed. On success a session cookie (HttpOnly, Secure, SameSite as configured) is issued and the analyst is routed to the dashboard. Access to attacks, incidents, AI investigation, reports and WAF controls is allowed only after successful authentication, and administrative functions require the admin role.

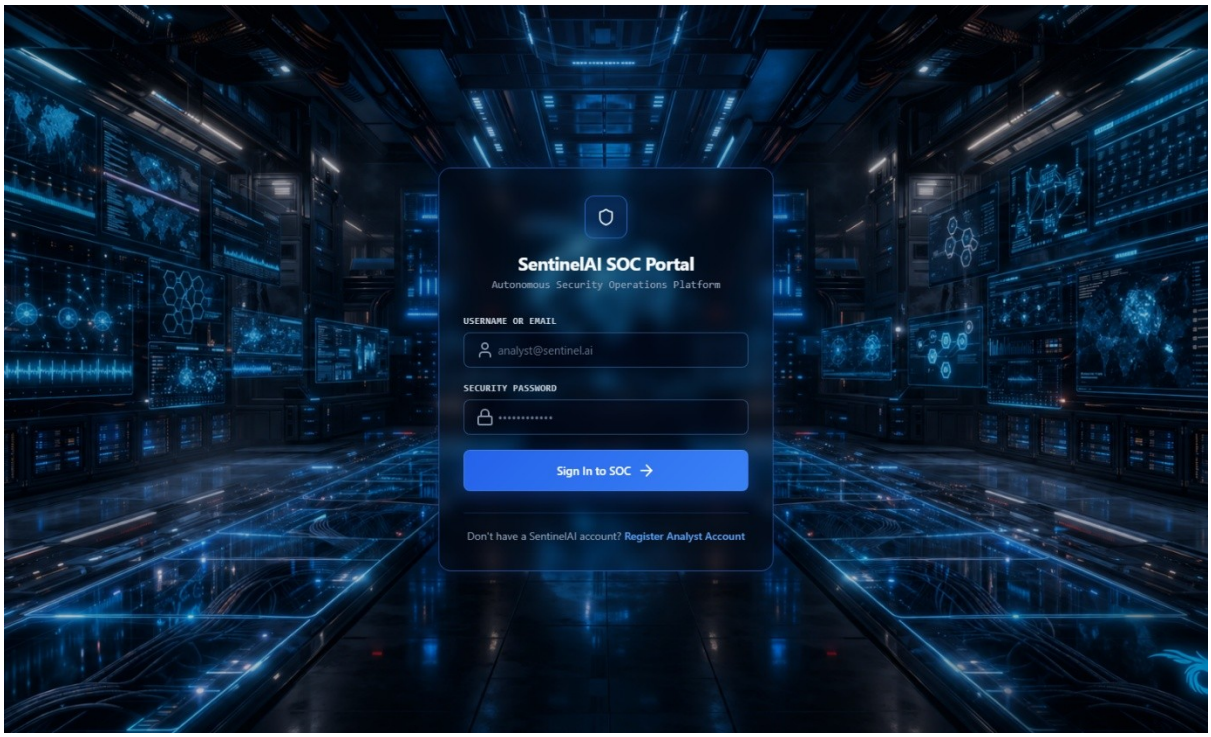


Fig. 6 SOC Analyst Login Portal

5.1.2 Real-Time SOC Command Center

The main overview page that answers the question: what is happening right now? It shows live host vitals (CPU, RAM, disk and uptime via psutil), a composite Threat Level (Critical, High, Medium, Low) with an AI confidence score, an interactive 3D Three.js threat globe projecting source-to-target vectors, attack-type distribution and trend charts, and a live feed updated over /api/attacks/ws.



Fig. 7 Real-Time SOC Command Center (Dashboard)

5.1.3 Live Threat Attack Feed & Triage

A detailed table of individual attack records with attack type, severity, source IP, timestamp, endpoint, protocol, detection source and status (NEW, INVESTIGATING, RESOLVED). Analysts can filter by severity, protocol (HTTP, SSH, FTP, TELNET), status and assignee, inspect payloads such as query strings, bodies, traversal attempts and SSH authentication attempts, and send any attack to the AI Investigator.

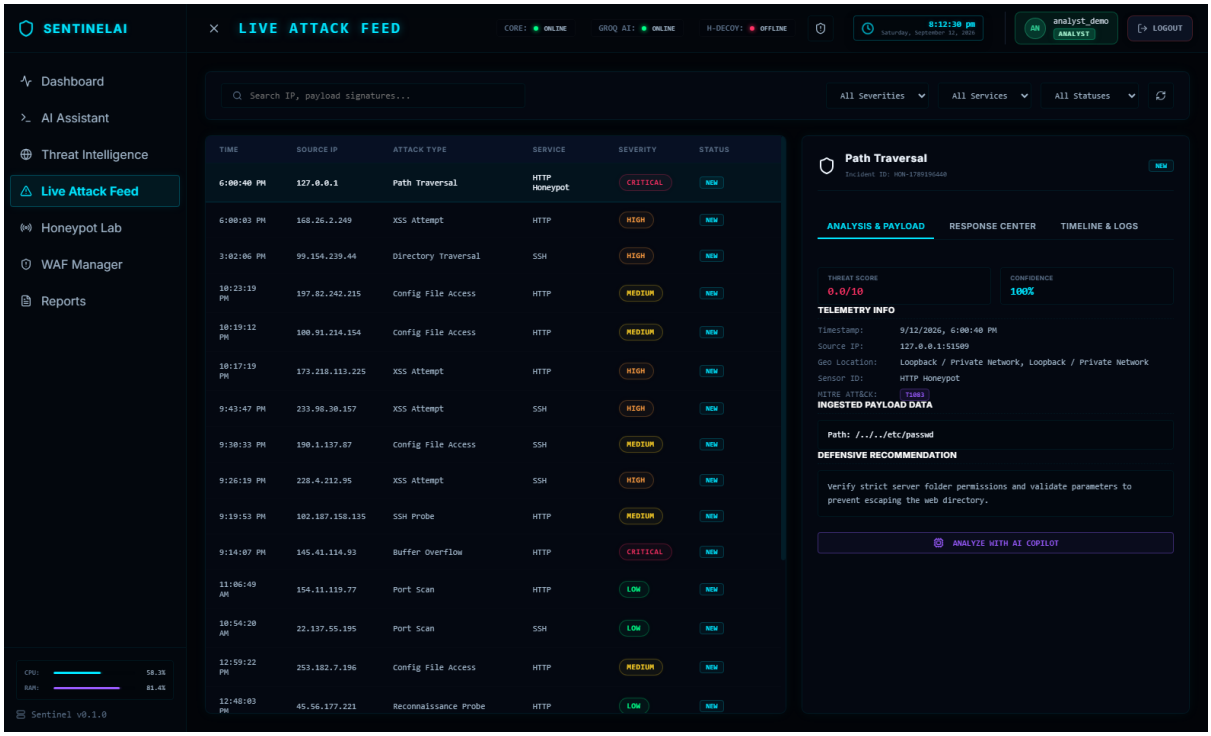


Fig. 8 Live Threat Attack Feed & Triage

5.1.4 AI Security Copilot & Investigator

A dual-mode workspace. The Telemetry Copilot tab offers a streaming chat interface for platform questions, with model status, provider and latency information and collapsed advanced settings. The Investigator tab provides an attack/incident selector, investigation timeline, context cards and the seven structured actions. When an attack is opened from the feed the context is injected automatically, so the analyst does not need to copy the IP address, payload or severity by hand.

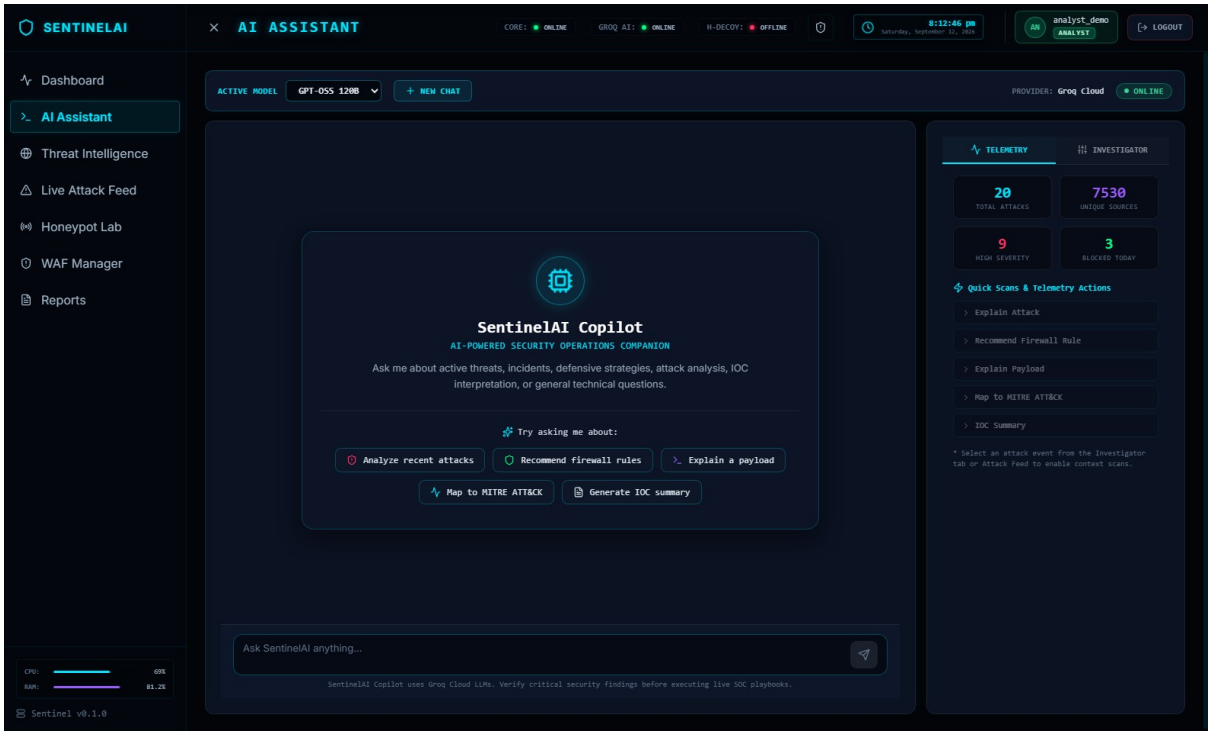


Fig. 9 AI Security Copilot & Incident Investigator

5.1.5 Threat Intelligence & Attacker Profiling

Aggregated attacker dossiers grouped by source IP, ASN, geographic origin and attack frequency. Each profile shows a dynamic 0–100 risk score, first-seen and last-seen times, targeted endpoints, techniques and related incidents, together with MITRE ATT&CK indicators such as T1110 (Brute Force), T1059 (Command Injection) and T1083 (File Discovery).

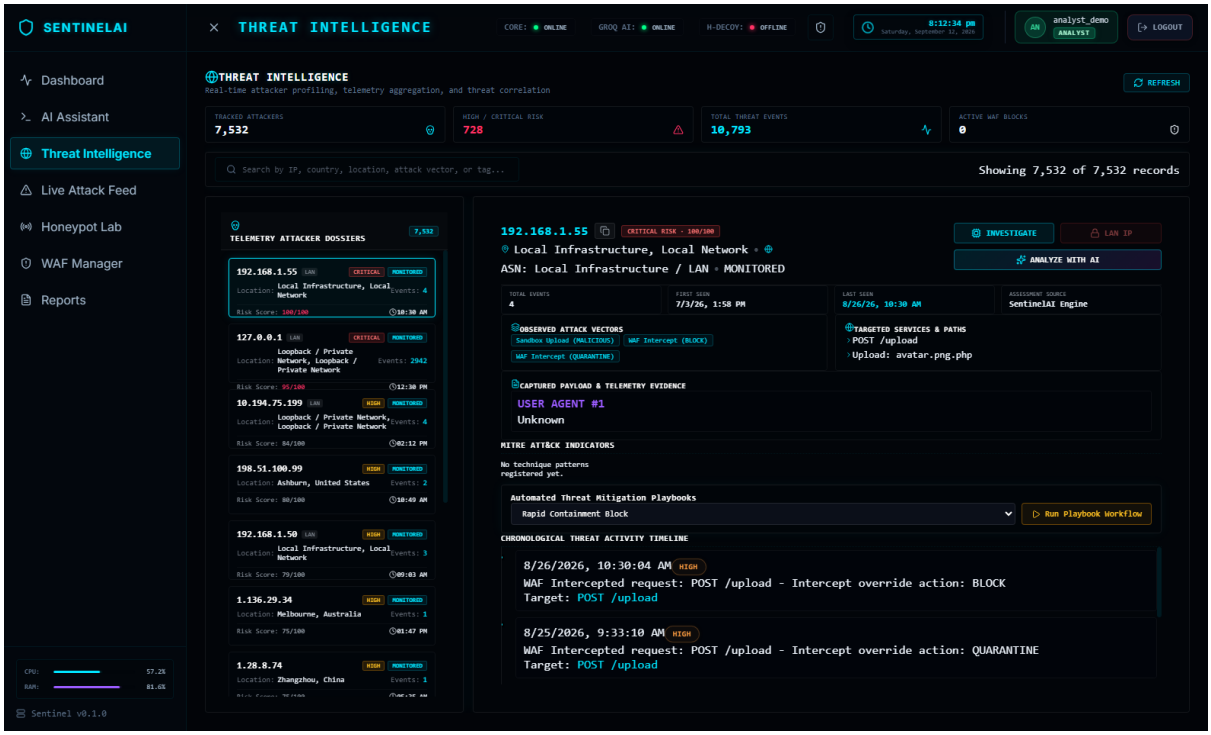


Fig. 10 Threat Intelligence & Attacker Profiling

5.1.6 Active Defense WAF Manager

The perimeter inspection console. It lists active rules and hit counts, enforces temporary and permanent IP quarantine with configurable expiry, and tracks observed sources (unique attackers, blocked requests and rule triggers). Requests matching a blocking rule receive a 403 response, a WAF hit is recorded and the event appears on the dashboard.

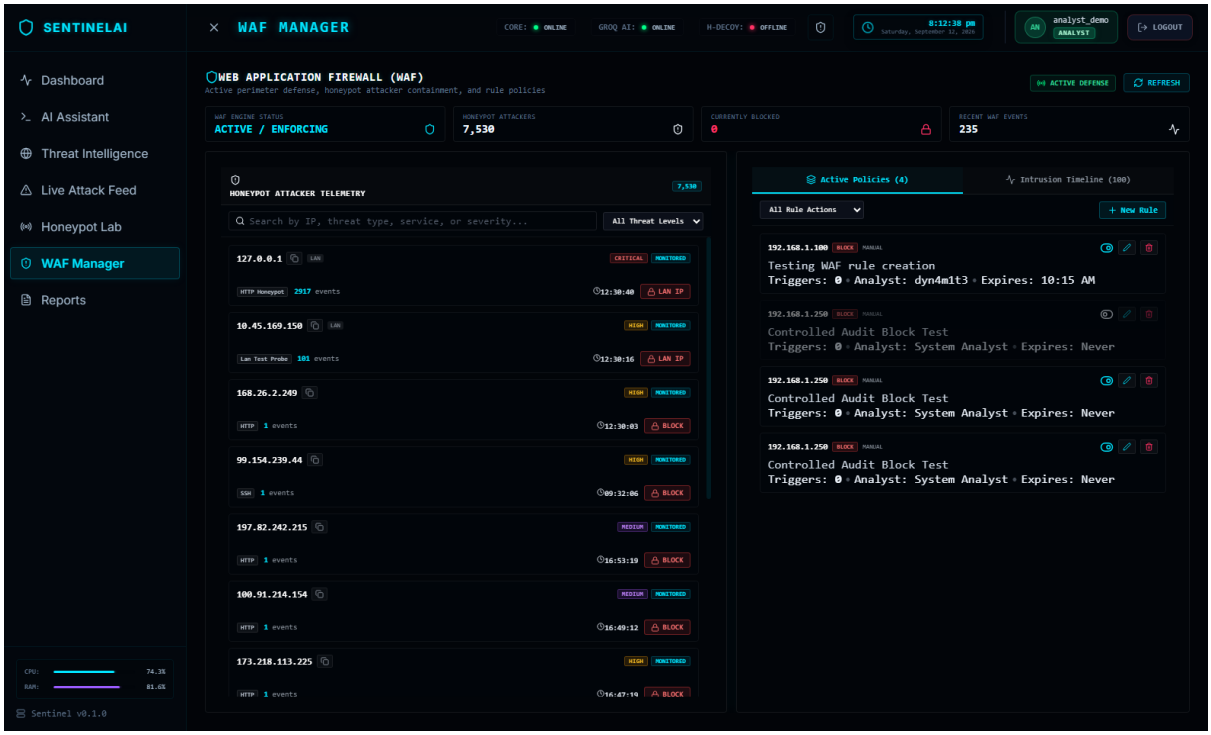


Fig. 11 Active Defense Web Application Firewall (WAF) Manager

5.1.7 Aetheris Decoy Honeypot Lab

Controls the multi-protocol decoys: an HTTP decoy portal (8088) and SSH (2222), FTP (2121) and Telnet (2323) listeners. The binding toggle switches between local loopback (127.0.0.1) for safe experimentation and LAN mode (0.0.0.0) for trusted-network telemetry capture. A live activity log shows requests as they arrive and a firewall-rule generator produces copy-ready PowerShell commands.

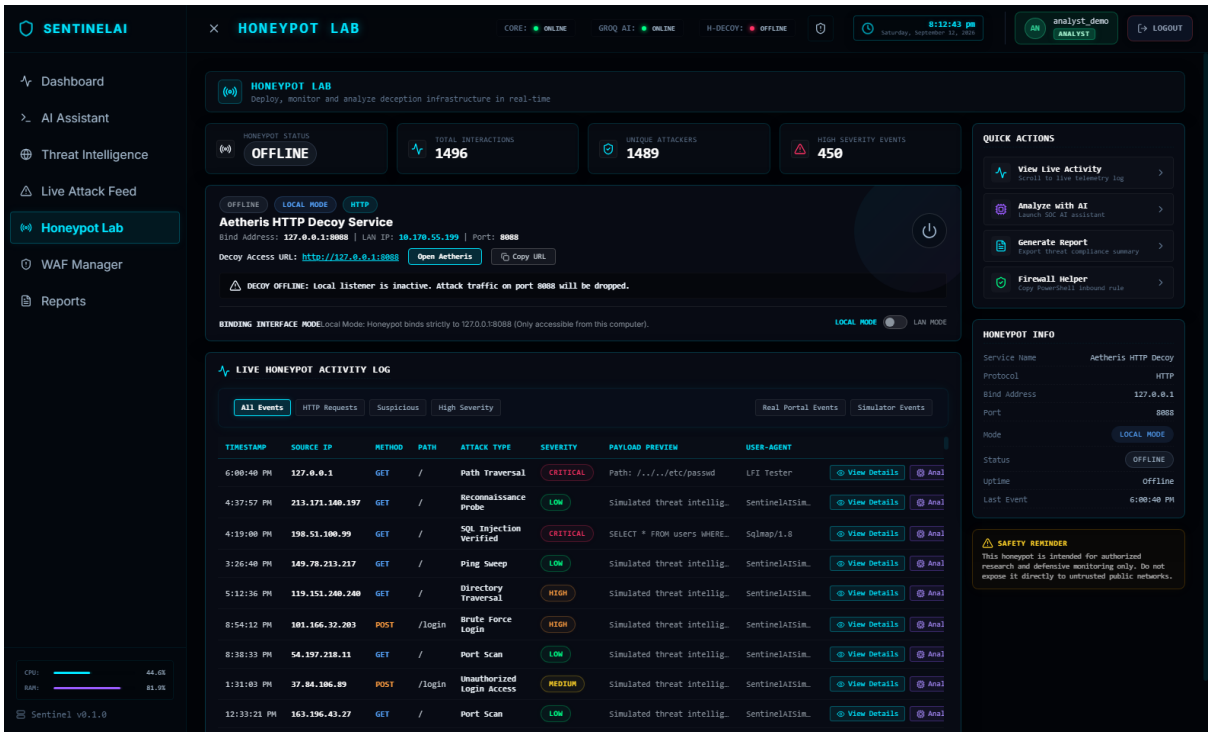


Fig. 12 Aetheris Decoy Honeypot Lab

5.1.8 Executive Reports & Export

Converts technical findings into documentation. The analyst selects a report type and range and exports an executive PDF containing summary, incident timeline, indicators of compromise, MITRE mapping, actions taken and remediation recommendations, or a CSV of raw telemetry for external SIEM or archival use. Reports are written to a persistent storage path.

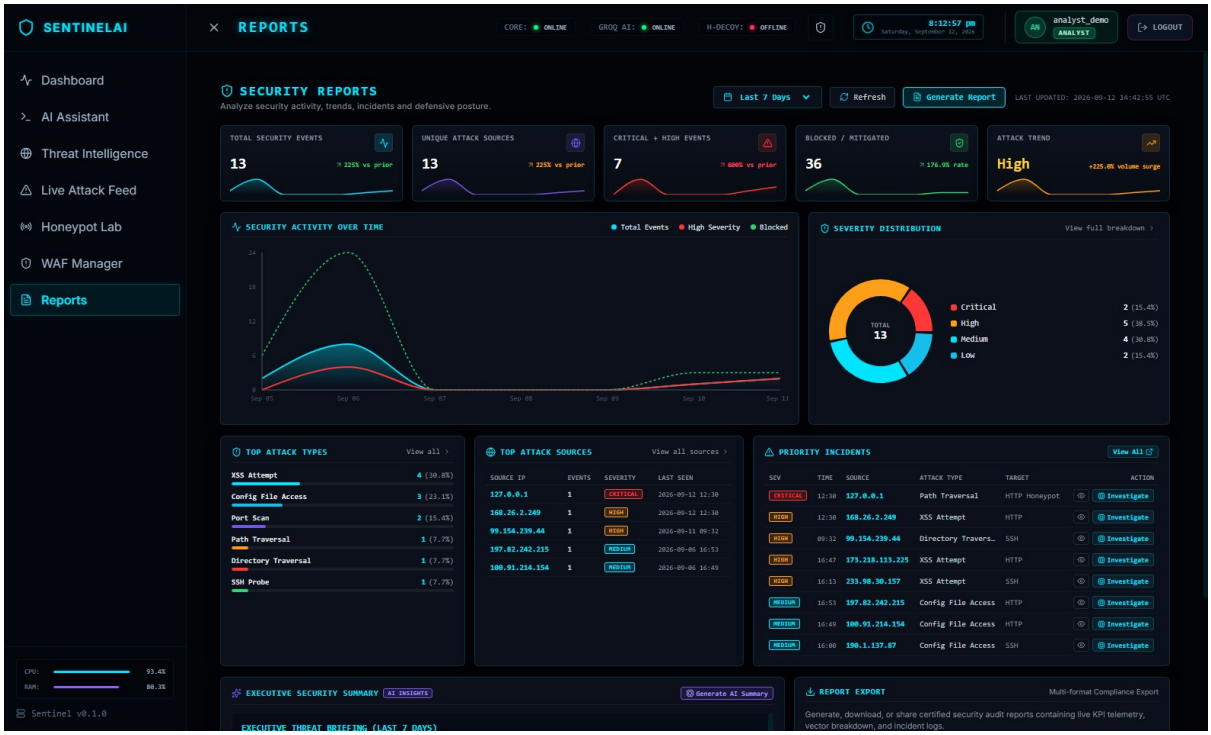


Fig. 13 Executive Compliance & Incident Reports

5.2 Test Results

5.2.1 Automated Backend Tests

The back end is verified by a Pytest suite of 51 tests covering API contracts, authentication, role-based access, WAF behaviour, honeypot logic, playbooks, reports, the decoy sandbox, the Groq adapter and PostgreSQL compatibility. At the Phase 16 documentation build the suite contained 19 tests (all passing); it has since grown to 51 passing tests with no failures. Observed warnings are non-blocking and relate to library behaviour.

Test module	Tests	Area
test_agent.py	8	AI Copilot and investigation actions
test_attacks.py	6	Attack records, feed and WebSocket manager
test_auth.py	7	Registration, login, session cookies
test_groq_agent.py	3	Groq provider adapter and fallback
test_honeypot.py	4	Decoy listeners and capture
test_playbooks.py	4	Remediation playbook execution
test_postgresql_compatibility.py	5	Dual-dialect database behaviour
test_rbac.py	4	Admin / analyst role enforcement
test_reports.py	4	PDF and CSV report generation
test_sandbox.py	3	Artifact hashing and heuristic scoring
test_waf.py	3	Rules, quarantine, blocking

Test module	Tests	Area
Total	51	51 passed, 0 failed

Pytest run (summary)

```
collected 51 items
backend/tests/test_agent.py ........                    [ 15%]
backend/tests/test_attacks.py ......                    [ 27%]
backend/tests/test_auth.py .......                      [ 41%]
...
backend/tests/test_waf.py ...                           [100%]
===== 51 passed, 575 warnings in 40.45s =====
```

5.2.2 Frontend Build Verification

The Vite production build completes successfully, which verifies React compilation, JavaScript bundling, dependency resolution and asset generation: 183 modules transformed, with the production bundle at roughly 492 kB (about 141 kB gzipped) and 38 kB of CSS. ESLint is run as part of the validation, and git diff --check reported no whitespace or formatting errors.

5.2.3 Functional Test Cases

#	Scenario	Input	Expected result	Verified by
1	SQL injection detection	' or 1=1 --	High-severity SQL injection attack	Pytest / simulator
2	XSS detection	<script>alert(1)</script>	XSS classification	Pytest / simulator
3	Repeated login failures	Many failures from one source	Brute-force alert	Pytest / simulator
4	Failure followed by success	Failures then successful login	Correlated account-compromise incident	Correlation logic
5	WebSocket delivery	New attack record	Dashboard updates without refresh	Manual (live dashboard)
6	AI context injection	Open attack via analyze_attack parameter	AI response references the selected attack	Pytest / manual
7	WAF block	Request matching a blocking rule	403 response and WAF hit record	Pytest
8	Report generation	Selected range, PDF format	PDF report generated	Pytest
9	Threat-intel failure	External API unavailable	Fallback result, no application failure	Pytest
10	Frontend build	npm run build	Successful production build	Build log

5.2.4 Evaluation Metrics

Metric	What it measures
Detection accuracy	Correct classification of known test payloads
False-positive rate	Benign inputs incorrectly marked as malicious
Event processing time	Arrival to storage and alert display
WebSocket latency	Backend event creation to frontend update
AI response latency	Time to first token and full analysis (SSE streaming)
Correlation accuracy	Whether related events are grouped correctly
System stability	Behaviour during service errors and external API failures
Build reliability	Whether the project builds consistently

5.3 Result Analysis

The result analysis for SentinelAI is as follows:

1. Complete lifecycle demonstrated

A single event travels from capture through detection, enrichment, normalisation, correlation, AI investigation, response and report, which is the principal result of the project. Many student security projects perform only one of these functions.

2. Explainable detection

Because detection is rule-based, every alert carries an attack type, severity, rule and evidence. The AI adds explanation but does not decide whether an attack occurred.

3. Incidents instead of isolated alerts

Ten failed logins, one success and access to a sensitive endpoint become one higher-priority case rather than three unrelated alerts, reducing alert fatigue.

4. Low-latency, streaming AI assistance

Groq-hosted inference with SSE streaming lets text appear as it is generated; the duplicated loading-state problem was corrected so that only streamed text is shown once output begins.

5. Resilience

Fallback logic for the AI provider and threat-intelligence lookups keeps the interface stable when external services fail.

6. Safe experimentation

Decoy listeners are passive, the sandbox stores artifacts without execute permission, and loopback and LAN modes are separated, supporting safe local demonstration.

5.3.1 Result Summary

The back end passes all 51 automated tests, the front end builds successfully, real-time WebSocket updates work, the Investigator workspace is stable at 100 percent browser zoom

and the deployment foundation (Docker, Compose, Nginx, PostgreSQL support, backup scripts) is committed to the repository. Compared with basic student systems, which each perform a single function, SentinelAI connects Event → Detection → Enrichment → Correlation → AI Investigation → Response → Reporting. Compared with enterprise platforms it is smaller and easier to understand, and its strength is educational completeness and architectural clarity rather than scale.

Known limitations. Docker-stack runtime validation and public production deployment are still pending; detection is rule-based (no anomaly models); AI output may be plausible but imperfect and is therefore presented as reviewable guidance; coverage is bounded by the quality of the rules and by the availability of external intelligence data; voice interaction and cross-network honeypot deployment are postponed.

5.4 Challenges Faced and Solutions

Challenge	Issue	Solution
Real-time synchronisation	Keeping dashboards current without polling	WebSocket broadcast of attack events
AI provider migration	Local model setup had performance and model-management limits	Moved to Groq Cloud with model discovery and chat-model filtering
Duplicate loading states	Loading card and streamed text shown together	Loader hidden when streaming text starts
Investigator sidebar clipping	Cramped at standard zoom	Constrained height, independent scrolling, adjusted width and spacing
Different event formats	Honeypot, simulator, WAF and auth events differ	Introduced a normalised log schema
Threat-intel availability	APIs fail or require keys	Deterministic fallback enrichment
SQLite vs PostgreSQL	Behavioural differences between dialects	URL-based engine options and dialect compatibility tests
Secret management	Deployment files must not hold credentials	Environment variables and placeholders; .gitignore for backups
WebSocket in production	Hard-coded localhost URLs fail	Environment-based, protocol-aware URL handling
Safe honeypot exposure	Honeypots must not create risk	Local/LAN modes, passive decoys, public exposure postponed

CHAPTER - 6
CONCLUSION & FUTURE SCOPE

6.1 Conclusion

Security teams are overwhelmed by raw events and fragmented tools, while students rarely see how a complete SOC pipeline works. To address this, SentinelAI was developed as an integrated, AI-assisted cyber defense platform that captures events from honeypots and simulators, detects common web attacks with transparent rules, enriches source addresses, normalises logs, correlates related events into incidents mapped to MITRE ATT&CK, explains them through a context-aware streaming AI Copilot, supports active defense and remediation playbooks, and documents the outcome in executive reports.

The platform is technically significant because it combines cybersecurity engineering, artificial intelligence, front-end and back-end development, database design, WebSocket and SSE communication, threat intelligence, incident response and deployment engineering. Its principal achievement is not a single feature but the complete lifecycle: **Collect → Detect → Enrich → Normalize → Correlate → Investigate → Respond → Report.**

The current build has a stable Investigator workspace, 51 passing automated tests, a successful production front-end build, documented architecture, real-time monitoring, Groq AI integration with a deterministic fallback, role-based access control with Argon2id and secure session handling, and a committed deployment foundation. SentinelAI is useful as an academic project, internship demonstration, portfolio platform, SOC training environment and controlled laboratory, and as a foundation for a more advanced custom security operations platform. It is not a replacement for commercial SIEM or XDR products and does not claim to be one.

6.2 Future Scope

- **Local Docker validation and migrations:** audit and run the Compose stack, and introduce Alembic for versioned schema migrations.
- **Stronger access control:** multi-factor authentication, rate limiting on login, AI, report and sensitive endpoints, and finer-grained roles (administrator, SOC analyst, investigator, read-only viewer).
- **Secure public deployment:** HTTPS with a domain, hardened firewall rules, secret rotation, verified backups and monitoring; a Tailscale or WireGuard tunnel connecting the private honeypot lab.
- **Anomaly detection and advanced correlation:** machine-learning models for deviations from normal behaviour and graph-based or temporal correlation with risk scoring.
- **Network and endpoint telemetry:** Zeek or Suricata integration and distributed eBPF sensor agents for endpoint events.
- **Threat-intelligence expansion:** VirusTotal, MISP, AlienVault OTX and GreyNoise.

- **SOAR and integrations:** configurable playbooks, webhook actions towards cloud firewalls (AWS Security Groups, Cloudflare WAF, Azure NSGs), ticketing (Jira, ServiceNow, GitHub issues) and chat notifications (Slack, Teams, Telegram, email).
- **SIEM forwarding and observability:** Syslog/OpenTelemetry exporters, Prometheus, Grafana, centralised logging and Sentry.
- **Multi-tenant workspaces and compliance reporting:** separate honeypot fleets and policies per organisation, with report templates for ISO 27001, NIST, SOC 2 and PCI DSS.
- **Model flexibility and voice:** multiple AI providers with local fallback models, and voice input/output for the assistant.
- **Container security and high availability:** image scanning, non-root users, read-only file systems, dependency pinning, multiple back-end instances and managed PostgreSQL.

Roadmap

Phase	Planned work
A — Local validation	Audit Docker files, build images, start the stack, verify database initialisation, API routing, WebSockets, report persistence, backup and restore
B — Database hardening	Introduce and test Alembic migrations; define and verify a backup policy
C — Security hardening	Generate secure secrets, implement rate limiting, review authentication, extend RBAC, run dependency scans
D — Staging deployment	Create staging server, configure HTTPS, deploy containers, test monitoring and rollback
E — Hybrid lab integration	Configure private tunnel, authenticate lab event ingestion, keep honeypot isolated, test private-to-public flow
F — Production readiness	Monitoring, incident alerts, backup automation, disaster recovery, security review, documentation update, release tag

REFERENCES

1. National Institute of Standards and Technology, "Computer Security Incident Handling Guide," NIST Special Publication 800-61.
2. National Institute of Standards and Technology, "Cybersecurity Framework."
3. MITRE Corporation, "MITRE ATT&CK Knowledge Base." Available: https://attack.mitre.org
4. OWASP Foundation, "OWASP Top 10 Web Application Security Risks."
5. OWASP Foundation, "Web Security Testing Guide."
6. OWASP Foundation, "ModSecurity Core Rule Set Documentation."
7. K. Scarfone and P. Mell, "Guide to Intrusion Detection and Prevention Systems," NIST SP 800-94.
8. K. Kent and M. Souppaya, "Guide to Computer Security Log Management," NIST SP 800-92.
9. NIST, "Zero Trust Architecture," Special Publication 800-207.
10. FastAPI Documentation. Available: https://fastapi.tiangolo.com
11. React Documentation. Available: https://react.dev
12. Vite Documentation. Available: https://vite.dev
13. SQLAlchemy Documentation. Available: https://docs.sqlalchemy.org
14. PostgreSQL Documentation. Available: https://www.postgresql.org/docs
15. Docker and Docker Compose Documentation. Available: https://docs.docker.com
16. Nginx Documentation. Available: https://nginx.org/en/docs
17. GroqCloud API Documentation. Available: https://console.groq.com/docs
18. AbuseIPDB API Documentation. Available: https://docs.abuseipdb.com
19. ip-api Geolocation Service Documentation. Available: https://ip-api.com/docs
20. The Zeek Project, "Zeek Network Security Monitor Documentation."
21. Open Information Security Foundation, "Suricata Documentation."
22. Cisco, "Snort Intrusion Detection and Prevention Documentation."
23. Cowrie Honeypot Project Documentation.
24. MISP Project Documentation; AlienVault Open Threat Exchange Documentation.
25. OWASP, "Authentication Cheat Sheet"; "Cross-Site Scripting Prevention Cheat Sheet"; "SQL Injection Prevention Cheat Sheet"; "REST Security Cheat Sheet"; "Secrets Management Cheat Sheet."
26. C. Sanders, Practical Packet Analysis. No Starch Press.
27. R. Bejtlich, The Practice of Network Security Monitoring. No Starch Press.
28. M. Collins, Network Security Through Data Analysis. O'Reilly Media.

29. A. Chuvakin, K. Schmidt and C. Phillips, Logging and Log Management. Syngress.

30. G. Vaishnav Kumar, "SentinelAI — AI-Powered Cyber Defense Platform," GitHub repository, 2026. Available: https://github.com/Vaishnav53/SentinelAI